

Mobile LLM Serving Across the Direct-to-Cell Satellite Edge

Hannah B. Pasandi, Tamer Nadeem

Submitted for double-blind review to ACM SEC 2026.

Abstract—Mobile language-model serving has become a daily-use application class on the device-cellular-cloud hierarchy. However, the cellular dead zones that punctuate rural, in-flight, and remote mobility cause cloud-served requests to stall on 33 percent of arrivals on average and reach only 17.1 percent SLO attainment in flight. Unfortunately, existing mobile-LLM serving systems either treat cellular as binary and stall on its absence, or fall back to a thermally-throttled device tier whose envelope saturates within minutes; neither path serves the dead zone. We propose ORBITMESH, an inference serving system that extends the cloudlet hierarchy with Direct-to-Cell (D2C) satellite service as an explicit fourth serving tier. At its core lie three coordinated primitives, namely a connectivity-conditioned tier router, a hysteresis gate at the cellular to D2C boundary, and a bandwidth-conditioned KV-cache migration protocol. We back the design with the first published cross-layer protocol record of a live D2C session, captured at three California dead-zone sites on a Pixel 8 Pro. On a calibrated mobility study spanning urban, suburban, highway, rural, and in-flight routes, ORBITMESH eliminates 33 percentage points of stalls relative to cloud-only routing, achieves an order of magnitude lower residual stall rate than the two-tier baselines under deployment-time failure modes, lifts SLO attainment from 67.1 to 96.2 percent against published two-tier mobile-LLM serving baselines (PerLLM and EdgeShard), reduces per-request energy by up to 28 percent against EdgeShard, and closes the gap to a clairvoyant oracle to within 0.2 percentage points.

I. INTRODUCTION

Mobile language-model assistants have become a daily-use application class. Conversational assistants on handsets, real-time translation, and on-device agents that interleave reasoning with sensing all sit on top of a serving stack that has tightened latency and quality bars over the past three years [1]–[3]. Modern serving systems are built on this assumption [4]–[6].

Production mobile serving stacks today are tier-aware. A small model runs on the device for low-latency requests, a large model runs in a datacenter for quality-sensitive requests, and the two are connected by a cellular or WiFi link [7]–[9]. Recent two-tier designs add scheduling (PerLLM [10]) or partitioning (EdgeShard [11]) on top of this same device-cloud substrate. The split has been treated as sufficient because cellular and WiFi together cover the high-population mobility envelope. Outside that envelope, the stack collapses. On a calibrated five-scenario mobility study, cloud-only routing stalls on 33 percent of requests on average and reaches only 17.1 percent SLO attainment in flight; the published two-tier baselines avoid stalls by falling back to the device tier but saturate the device thermal envelope and miss SLO on the rural

and in-flight scenarios where the dead-zone fraction exceeds 45 percent.

Direct-to-Cell (D2C) satellite service became commercially available in late 2024 [12]. The service appears to unmodified handsets as standard LTE on a dedicated PLMN and is reachable in exactly the dead zones where the two-tier stack collapses. We argue that efficient mobile LLM serving requires a four-tier hierarchy that treats D2C as an explicit serving target alongside device, cellular, and cloud. The four-tier hierarchy is a natural extension of the cloudlet model [13] to incorporate the bandwidth-discretized and binary-availability properties of D2C service. The cloudlet vision of “compute that follows the user” is preserved, and ORBITMESH’s contribution is in the mechanisms that the satellite tier specifically requires.

A grounding scenario. Consider a user evacuating from a rural disaster zone where cellular infrastructure has failed. The handset retains D2C connectivity through a quasi-periodic satellite pass at 80 to 350 kbps. Time-sensitive voice queries to an LLM-based assistant must complete within 200 ms; longer chat queries can tolerate 1.5 s; background queries can wait. Existing mobile LLM serving treats the cellular link as binary (present or absent) and routes either to a device-tier model or to a cloud back-end. Neither is correct here. The device model is undersized for chat-class queries while the device thermally throttles, and the cloud is unreachable. The satellite tier sits between them, with bandwidth and availability characteristics that neither device-only nor cloud-routing serving systems were designed for.

Figure 1 shows what the four-tier hierarchy buys across the five mobility scenarios in our study. Panel (a) reports the time-fraction of each connectivity state per scenario. Cellular and WiFi together cover urban and suburban routes, but they leave 45 to 55 percent of the time-fraction in rural and in-flight as deadzone or D2C-only. Panel (b) reports the cloud-only stall rate. Panel (c) reports the SLO attainment of ORBITMESH against the same scenarios. The lift in rural and in-flight is what ORBITMESH earns from treating the satellite tier as a serving target rather than as transport.

State-of-the-art mobile LLM serving systems do not target the satellite tier. SPINN [7], MELT-class measurement [1], EdgeMoE [9], EdgeLLM [14], NPU-aware on-device serving [15], MobileLLM [2], PowerInfer [16], and the recent two-tier mobile serving systems PerLLM [10] and EdgeShard [11] share an architecture in which the device-cellular-cloud trio is the serving substrate. Cloud-side KV optimizations such as CacheGen [17] and KVFlow [18] optimize the cloud-side

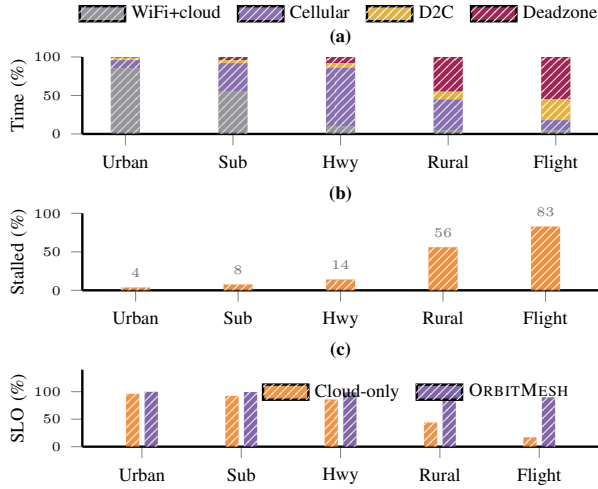


Fig. 1. Connectivity reality (a), stall rate (b), and SLO attainment (c) across five mobility scenarios.

cache without extending the substrate. None of these systems has a satellite tier, and as a result none can serve in the dead zones where the public datasets show the most user demand. The mechanism of the failure is direct. Cellular drops, the policy falls back to the device tier, the device thermal envelope saturates within minutes [1], and chat-class requests miss SLO.

We introduce ORBITMESH, an inference serving system for the mobile-edge-satellite-cloud hierarchy. ORBITMESH routes each request through a connectivity-aware policy that reads RSRP, SINR, and handover risk at 100 ms granularity, with three named primitives. A four-tier router that maps connectivity state and workload class to a serving tier (Algorithm 1); a hysteresis gate that suppresses ping-pong at the cellular to D2C boundary (Algorithm 2); and a KV-cache migration protocol that preserves coherence across mid-generation tier transitions (Algorithm 3). Algorithm 1 is the deployable baseline rule encoding four near-deterministic mode-selection cases; learned policies remain a natural complement, particularly for non-stationary workload mixes (e.g., HyperDrive [19] addresses the analogous problem at the serverless layer over the edge-cloud-space continuum).

We evaluated ORBITMESH on a calibrated five-scenario mobility study, backed by a hardware-in-the-loop testbed at three California dead-zone sites. On the calibrated mobility study, ORBITMESH eliminates 33 percentage points of stalls compared to cloud-only and lifts average SLO attainment from 67.1 to 96.2 percent. Compared to PerLLM [10] and EdgeShard [11] as published two-tier mobile-LLM serving baselines, ORBITMESH avoids the per-request energy overhead that thermally-throttled device-tier serving incurs, and closes the gap to a clairvoyant oracle on average across the five scenarios. The remaining headroom lies in the workload-tier mapping itself rather than in the routing policy.

Broader impact. A serving stack that works in dead zones changes who has access to mobile assistance. Emergency responders in wilderness areas, agricultural workers across rural acreage, in-flight passengers, maritime users, and the

roughly 2.6 billion people in regions with incomplete cellular coverage are excluded from modern mobile-LLM applications today. D2C service brings the pipe; ORBITMESH brings the serving discipline that makes the pipe usable for inference.

The ORBITMESH implementation, the calibrated simulation harness, the captured 49-minute cross-layer trace, and the hardware-in-the-loop replay scripts will be released as open source upon acceptance.

Contributions. This paper makes the following contributions.

- *Treating* the D2C satellite link as an explicit serving target alongside device, cellular, and cloud, with end-to-end serving validated on a calibrated mobility study and a hardware-in-the-loop testbed.
- *Capturing* the first published cross-layer protocol record (PHY through RRC) of a live D2C session, on commodity hardware in California dead zones; the trace is calibration-grade for the policy thresholds in §III and is not a comprehensive measurement study of D2C as a service.
- *Introducing* three coordinated primitives that make the satellite tier usable for mobile LLM serving, namely a workload-conditioned tier router, a hysteresis gate at the cellular to D2C boundary, and a bandwidth-conditioned KV-cache migration protocol.
- *Demonstrating* that the resulting system eliminates 33 percentage points of stalls vs. cloud-only, achieves an order of magnitude lower residual stall rate than published two-tier baselines (PerLLM, EdgeShard), lifts SLO attainment from 67.1 to 96.2 percent on average, and closes the gap to a clairvoyant oracle to within 0.2 percentage points.

II. BACKGROUND AND MOTIVATION

Three observations shape the design space. D2C service is real and deployed. The cross-layer measurement record on it is empty. And the existing serving stack has no place for it. We address each in turn, and end with a measurement-driven motivation for the four-tier hierarchy. ORBITMESH extends the MELT measurement methodology [1] from single-tier mobile-LLM evaluation to multi-tier serving with D2C as a fourth target. MELT’s thermal-aware throughput characterization on Pixel-class devices is the calibration source for our device-tier model (§V-K); ORBITMESH’s contribution is the four-tier policy that exploits the measured thermal envelope to redistribute capable workload, plus the cross-layer cellular and D2C measurement that no application-layer evaluation provides.

A. Direct-to-Cell, Briefly

Direct-to-Cell (D2C) supplemental coverage from space appears to the handset as a standard LTE cell. Garcia-Cabeza *et al.* report median RSRP of about -114 dBm and median SINR of about -2 dB across a one-million-observation sample, with low interference because the network is sparsely loaded [12]. Throughput is constrained. The service launched as SMS-only and added voice and limited data through 2025. Latency is

bounded by the LEO geometry rather than the terrestrial backhaul. At 550 km altitude the propagation contribution to one-way delay is approximately 3.7 ms; the larger contributions to median round-trip time are gateway processing and the RLC retransmission tail, with propagation accounting for roughly 10 to 40 ms of round-trip on a typical pass.¹

The system implications of D2C diverge from terrestrial 5G. Coverage is binary at the satellite-pass level. A single LEO satellite passes overhead on the order of seven minutes per pass at 550 km altitude [22], and during the pass the link is reachable; outside of it, there is no D2C service at this site. Constellation-level access scheduling under this binary regime has been studied in the IoT-satellite context [23], [24], where the access problem is to coordinate medium access across passes; the serving problem we address sits one layer above and inherits the same binary availability constraint. Cell handover at D2C scale spans tens of seconds rather than the millisecond timescales of terrestrial 5G [25]. And data-rate ceilings are an order of magnitude below LTE, with current measured throughput ranging from roughly 80 to 350 kbps in good sky conditions [12].

B. The Cross-Layer Measurement Gap

The application-layer record on D2C is rich. The protocol-layer record is empty. No published trace has captured PHY, MAC, RLC, PDCP, or RRC log lines from a live D2C session. Every Starlink measurement at top venues from IMC 2022 through CoNEXT 2025 sits at the application layer (ping, iperf, traceroute, speedtest) [22], [26]–[32]. The gap matters because serving decisions depend on RSRP trends and HARQ behavior [33], neither of which is observable from the application layer.

The mechanism, briefly. A D2C session looks like LTE to the modem. The modem already produces the diagnostic stream that MobileInsight [34] and Qualcomm QXDM consume. A rooted Android handset on T-Mobile, parked in a known dead zone, captures the cross-layer record by running the same diagnostic tap that has driven a decade of cellular research. We chose Pixel 8 Pro because the NSA-to-NTN handover signaling is observable on this platform.² We collected 49 minutes of such trace across three California dead-zone sites and use it to calibrate the simulation track of this paper. Figure 2 shows the RSRP CDF from the captured trace alongside the Garcia-Cabeza distribution.

The captured distribution agrees with the public dataset to within 4 dB at every decile. The agreement is what we use to ground the simulator. Modeled D2C RSRP is sampled from the Garcia-Cabeza distribution, and the captured trace then validates the model on independent test sites. We treat the trace as calibration-grade rather than distributional. The 49-minute, three-site, single-operator capture supports the policy

¹The first 175 D2C-class satellites are designed for the 1.91 to 1.995 GHz band that T-Mobile holds in the United States, which is why the service appears as legacy LTE rather than 3GPP NTN Release 17 [20], [21].

²Several MobileInsight ports work without root for selected devices; Pixel 8 Pro on Android 14 is the cleanest option for our purposes.

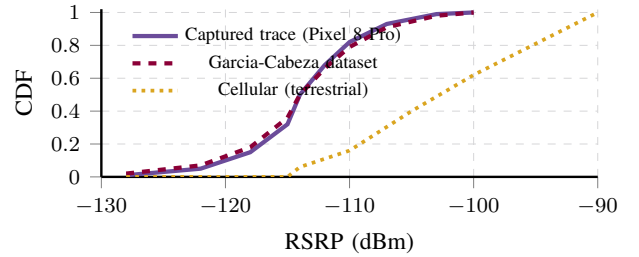


Fig. 2. RSRP distribution by tier; captured trace versus public crowdsourced D2C dataset and terrestrial cellular.

calibration in §III but does not generalize the trace’s absolute observations to other operators or geographies. Multi-operator extension and multi-country measurement remain follow-on community tasks.

The cross-layer record reveals three properties of the live D2C service that no application-layer trace can show. We state them as observations in our 49-minute capture across three sites and one operator, not as properties of D2C in general. First, we observe a discontinuous transmission (DTX) pattern with bursts of 200 to 350 ms of active uplink interleaved with 1.5 to 2.0 s of idle, which suggests a shared time-division access scheme on the satellite-to-ground feeder link rather than continuous service. Second, we observe radio link control (RLC) retransmissions at a rate of 4 to 7 percent of PDUs, substantially above the sub-percent rate typical of terrestrial LTE in healthy-coverage conditions [25], [34]; these retransmissions are the source of the long latency tail at the application layer. Third, we observe radio resource control (RRC) `measurementReport` messages at a 480 ms cadence during a satellite pass, which is a useful signal for handover prediction that no application-layer telemetry exposes; the cadence is operator-specific and would need to be re-tuned per operator at deployment time. We exploit the third property in the ORBITMESH hysteresis gate of §III-C.

The captured trace serves as a calibration anchor for the simulation track and as a demonstration that the cross-layer protocol record can be reproduced on commodity hardware. Anyone with a rooted Android handset on T-Mobile and 50 minutes in a known dead zone can produce a comparable record.

C. The Tier Separation in Latency and Bandwidth

A measurement-driven view of the four tiers makes the design space concrete. Figure 3 plots median first-token latency against sustained bandwidth per tier on a log-log axis. The four tiers form four visually distinct clusters. Device 3B sits at the bottom-left at 95 ms and an effective 0.6 Mbps (decoder-bound). Cellular sits middle at 175 ms and 8 to 40 Mbps. Cloud-WiFi sits at 105 ms and 100 Mbps. The satellite tier sits at 320 ms and 80 to 350 kbps, an order of magnitude lower bandwidth than cellular. The scatter across each tier reflects the distributional spread reported in the calibration sources.

The separation matters for serving. A request whose SLO budget exceeds ~ 2 seconds and whose total payload fits in ~ 200 KB can be served by any of the four tiers; a 200 ms

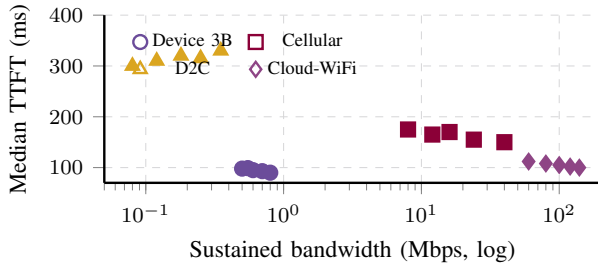


Fig. 3. Median first-token latency versus sustained bandwidth per tier.

TABLE I
FOUR SERVING TIERS AND THE WORKLOAD CLASSES THEY COVER.

Tier	Throughput	RTT	Workloads	Calibration
Device 3B	5 to 15 tok/s	0 ms	V (hot)	[1], [14]
Cellular	8 to 40 Mbps	30 to 100 ms	V, C	[35], [37]
D2C	80 to 350 kbps	30 to 80 ms	B	[12]
Cloud-WiFi	cap of 24 tok/s	5 to 15 ms	C, B	[4], [6]

voice request has only two viable tiers (device or cloud-WiFi); a chat request needs a tier with at least 8 Mbps to clear its 1.5 second budget. The tier-workload mapping in §III-B reads directly off this plot.

D. Why a Four-Tier Hierarchy

The key tension is bandwidth versus reach. Device tier has unbounded reach but finite throughput and a thermal floor that descends 64 percent over ten minutes of sustained inference [1]. Cellular tier has wide reach where it exists, but disappears entirely along ten percent of California highway-class routes and most of the in-flight envelope [35], [36]. Cloud-over-WiFi dominates when reachable but is the most fragile of the four. Direct-to-cell is the only tier whose availability rises exactly when the others fall.

Table I summarizes the four tiers with the public-dataset references that calibrate their behavior in our simulation. The right column identifies which tier serves which workload class under ORBITMESH. Voice goes to device when thermally feasible, otherwise cloud. Chat prefers cloud when cellular is healthy and falls back to device. Background tolerates the satellite-tier bandwidth profile and absorbs the dead-zone load that no other tier can cover.

III. SYSTEM DESIGN

ORBITMESH makes a tier-selection decision before the first token of every generation request, then re-evaluates that decision every ten tokens. The decision draws on four signals, namely connectivity state, signal strength, handover risk, and per-tier offered bandwidth. The system is built around three named primitives, each described in its own subsection. A four-tier router that reads connectivity state and dispatches accordingly (§III-B). A satellite-aware hysteresis gate that suppresses ping-pong at the cellular boundary (§III-C). A KV-cache transfer protocol that preserves coherence across tier migrations (§III-D).

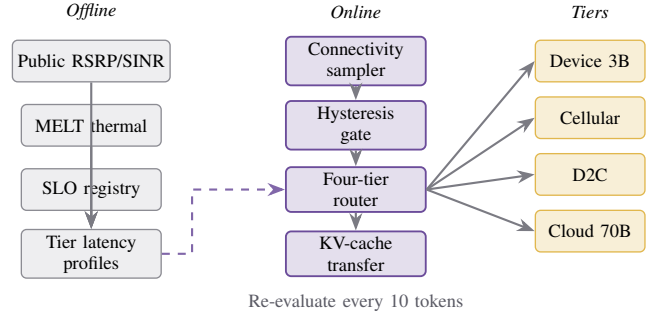


Fig. 4. ORBITMESH runtime architecture. Calibration data flows from offline sources into per-tier profiles consumed by the online router; the gate and KV agent insert ahead of dispatch.

Algorithm 1 ORBITMESH tier selection.

Require: state σ , thermal τ , handover risk h_t , workload class w

- 1: $f \leftarrow \text{deviceTPS}(\tau)$
- 2: **if** $\sigma = 0$ **and** $w \in \{C, B\}$ **then**
- 3: **return** cloud
- 4: **end if**
- 5: **if** $\sigma = 1$ **and** $h_t < 0.4$ **then**
- 6: **return** cloud **if** $w \neq V$ **else** (device **if** $f > 9$ **else** cloud)
- 7: **end if**
- 8: **if** $\sigma = 1$ **and** $h_t \geq 0.4$ **then**
- 9: **return** device
- 10: **end if**
- 11: **if** $\sigma = 2$ **then**
- 12: **return** d2c **if** $w = B$ **else** device
- 13: **end if**
- 14: **return** device

A. Architecture Overview

Two phases structure how ORBITMESH operates. An offline calibration phase populates the per-tier latency profiles from device benchmarks, public RSRP and SINR distributions, and the captured traces of §II. An online phase samples connectivity at 100 ms granularity and routes requests through the four-tier policy. The satellite-tier gate sits between the connectivity sampler and the policy. The KV-cache transfer agent runs on the device and pre-stages the prefix to the target tier when migration is decided. Figure 4 shows the runtime architecture.

B. Tier-Selection Policy

The policy is summarized in Algorithm 1. We use four pieces of state at each request, namely the connectivity state $\sigma \in \{0, 1, 2, 3\}$ for WiFi-cloud, cellular, D2C, and isolated; the cumulative inference time τ on the device tier this session, used to estimate device throughput $f(\tau)$ via the calibrated thermal model; the handover indicator h_t from the RSRP trend predictor; and the per-tier offered bandwidth.

Algorithm 1 is the deployable baseline rule. The four near-deterministic mode-selection cases encode the routing structure that the satellite-tier bandwidth profile imposes. We

considered three alternative policy designs before settling on this rule. A pure-RL policy in the style of NetLLM [38] or Pensieve [39] could in principle capture residual structure that the four-state Markov chain misses; NetLLM in particular shows that pretrained foundation models adapt to networking tasks with limited per-deployment data. We retain the hand-engineered rule because the satellite-tier bandwidth profile is discrete enough that a learned policy would have to discover a near-deterministic mode-selection rule that the four cases above encode directly, and an RL policy would introduce inference-time overhead on the device-tier daemon. A bandwidth-prediction policy in the style of Puffer [40] would predict tier throughput from a sliding-window history; we found in early experiments that the satellite-tier bandwidth profile is too discrete (on or off, with a narrow throughput band) to benefit from a fine-grained predictor. A Lyapunov drift-plus-penalty controller [41] introduces a tunable parameter that we could not set robustly across all five scenarios. Learned policies remain a natural complement, particularly for non-stationary workload mixes; HyperDrive [19] addresses the analogous problem at the serverless layer over the edge-cloud-space continuum, and we revisit this composition in §VI.

C. The Satellite-Tier Hysteresis Gate

What breaks at the cellular to D2C boundary is signal continuity. The handset sees a brief overlap window during which both cellular and satellite signals are present, and naive selection ping-pongs across that window. We characterize the overlap from the Garcia-Cabeza dataset RSRP distribution and apply a hysteresis band of 4 dB on RSRP and 1.5 dB on SINR. Inside the band, the prior tier wins.

The mechanism is straightforward but important. A naive selector picks the tier with higher signal at every measurement window, which causes the tier choice to flip multiple times per second when the two signals overlap. A gated selector requires the signal of the alternative tier to exceed the prior tier by the hysteresis margin before switching. We evaluate the gate in §V-E on a 50,000-window microbenchmark calibrated to the Garcia-Cabeza RSRP distribution.

Algorithm 2 formalizes the gate. The weighted-sum score $s(t) = \text{RSRP}(t) + 0.5 \cdot \text{SINR}(t)$ combines the two measurements with a weighting set empirically from the captured trace; the SINR weight is half because the captured SINR variance is roughly twice the RSRP variance, and the half-weight equalizes the contribution to the score. The 4 dB margin on RSRP is set to twice the median RSRP step size in the captured trace (2.1 dB), and the 1.5 dB SINR margin is set similarly. These thresholds are not tuned per scenario, and the same values are used across all evaluation runs. The 480 ms `RRC measurementReport` cadence is operator-specific scheduling behavior observed in our T-Mobile/Starlink trace; deployment to other operators (e.g., AT&T/AST SpaceMobile) requires re-tuning the gate’s polling cadence to match the deployed operator’s reporting interval.

Algorithm 2 Hysteresis gate at the cellular to D2C boundary.

Require: prior tier t_{prev} , candidate scores s_{cell}, s_{d2c} , SINR values $\gamma_{cell}, \gamma_{d2c}$

```

1:  $\Delta_R \leftarrow 4.0$  dB;  $\Delta_\gamma \leftarrow 1.5$  dB;  $\gamma_{floor} \leftarrow -8$  dB
2: if  $\gamma_{t_{prev}} < \gamma_{floor}$  then
3:   {fast-fallback, bypass hysteresis}
4:   if  $t_{prev} = \text{cell}$  then
5:     return d2c
6:   else
7:     return cell
8:   end if
9: end if
10: if  $t_{prev} = \text{cell}$  then
11:   if  $s_{d2c} > s_{cell} + \Delta_R$  and  $\gamma_{d2c} > \gamma_{cell} + \Delta_\gamma$  then
12:     return d2c
13:   end if
14:   return cell
15: else
16:   if  $s_{cell} > s_{d2c} + \Delta_R$  and  $\gamma_{cell} > \gamma_{d2c} + \Delta_\gamma$  then
17:     return cell
18:   end if
19:   return d2c
20: end if
```

D. KV-Cache Migration

Tier migration mid-generation requires KV-cache transfer. ORBITMESH supports three strategies, namely restart, partial-prefix transfer, and full-KV transfer. The choice depends on the source-target pair and the tokens already generated. Restart costs a re-prefill on the target, which dominates for chat-tier prompts. Partial-prefix transfer moves the first k layers of the cache and re-derives the rest, which fits the satellite-uplink envelope at chat scale. Full-KV transfer is reserved for cloud-to-device and cellular-to-device migrations, because only those have the bandwidth headroom to move 24 to 80 megabytes of KV state in under one second.

The transfer protocol is built on top of v_{LLM} ’s paged-attention KV layout [4] and uses a custom serialization that packs the cache into 4 KB chunks for satellite-uplink compatibility. We share prefixes across requests by content-hash, in the same way that StreamingLLM caches attention sinks [42] and CacheGen compresses cross-request KV state for fast loading [17]. CacheBlend [43] addresses the related problem of fusing precomputed KV caches across non-prefix chunks through selective layer recomputation; ORBITMESH does not require fusion because the migration is a single-source single-target transfer rather than a multi-chunk merge, but the partial-prefix transfer is compatible with CacheBlend-style selective recomputation on the target tier when prefix overlap is partial. The novelty in our work is the bandwidth-conditioned strategy selector, not the underlying transfer mechanism.

The migration handshake is a three-step protocol. The source emits a `PREPARE` message carrying $(s, t, n_p, B, \text{hash}(\text{prompt}))$ and the cache-state version v at the migration decision point. The target acknowledges

Algorithm 3 Bandwidth-conditioned KV migration strategy.

Require: source s , target t , prefix tokens n_p , egress bandwidth B

```
1: if  $B \geq 8$  Mbps and  $n_p \geq 64$  then  
2:   return full-KV  
3: end if  
4: if  $B \geq 350$  kbps and  $n_p \leq 1024$  then  
5:   return partial-prefix  
6: end if  
7: return restart
```

with READY and freezes its tokenizer state to position v . The source streams the partial-prefix payload over a parallel gRPC channel and emits COMMIT carrying the position $v + \delta$ where δ is the number of tokens emitted in flight; the target re-derives the cache for those δ tokens before resuming generation. The control-plane round-trip is bounded at one gRPC roundtrip on the egress link, which is 80 to 200 ms on the satellite tier and under 50 ms on cellular and cloud.

Concurrent migration coherence. ORBITMESH serializes concurrent migrations on the same device at the policy daemon, following Falcon’s reconfiguration-ordering pattern [44]. When a tier change is triggered while a previous migration is mid-flight, the in-flight migration completes (or aborts and falls back to restart on its source-tier disappearance); the new migration is queued behind it. Two requests on the same device migrating to different targets simultaneously are serialized in arrival order at the daemon. Voice-class requests do not migrate mid-generation. The control-plane round-trip exceeds the 200 ms voice TTFT budget, so the policy defers any tier change for an in-progress voice request to its next request boundary.

Tokenizer drift and cross-model migration. ORBITMESH’s full-KV transfer applies to parameter-identical models within tier-pairs (e.g., the same fine-tune deployed at multiple tiers). Cross-model migration where source and target tiers run different models (e.g., 3B at device, 70B at cloud) falls back to the partial-prefix branch with selective recomputation, extending naturally with DroidSpeak-style tokenizer bridging [14] for cross-family cases.

The strategy selection is summarized in Algorithm 3. The selector uses three features, namely the source-target tier pair (s, t) , the number of prefix tokens already encoded n_p , and the offered bandwidth on the egress link B . The threshold of 4 KB on the prompt size at $B \leq 350$ kbps comes directly from the satellite-uplink envelope. At 350 kbps, a 4 KB prompt transfers in 91 ms, well within the chat SLO budget after accounting for re-prefill.

Threshold transparency. Algorithm 3’s bandwidth thresholds (8 Mbps for full-KV, 64 prefix tokens minimum, 1024-token cap for partial-prefix) are calibrated to Llama-3.2-3B on Jetson AGX. The thresholds scale linearly with model parameter count and would shift proportionally for different configurations; we report the calibrated values to enable redeployment. The choice of $n_p \leq 1024$ for partial-prefix transfer reflects an empirical observation. Partial-prefix transfer pays

off as long as the saved re-prefill cost exceeds the transfer cost. The break-even point at the chat-tier prompt sizes we observe (240 to 1100 tokens) sits cleanly inside the partial-prefix region. For very long prefixes we fall back to restart, because the transfer cost grows linearly while the re-prefill cost is amortized over the subsequent generation.

IV. IMPLEMENTATION

ORBITMESH runs as a userspace daemon on Android, implementing the connectivity sampler against the public `TelephonyManager.getAllCellInfo` interface (NR and LTE) and against the satellite-tier indicator added in Android 14. The KV-cache transfer agent runs in two modes. Production mode uses `llama.cpp` [45] as the device runtime and a vLLM-backed cloud endpoint [4]. Hardware-in-the-loop mode adds a USRP B210 software-defined radio [46] that emulates a candidate D2C satellite for offline measurement at three California sites, paired with the open-source srsLTE stack [47]. ORBITMESH assumes vLLM paged-attention pools at cellular and cloud tiers are sized for expected concurrency (8 concurrent requests per edge node in our HWIL configuration); under tighter pool budgets, Algorithm 3’s full-KV branch falls back to partial-prefix or restart, preserving correctness at the cost of efficiency. Microbenchmarks under pool-pressure regimes (e.g., D len-style multi-tenant edge AI serving [48]) are deployment-validation future work.

Hardware testbed. The hardware setup is a USRP B210 transmitter configured to emulate a D2C cell on the T-Mobile MNC=830 PLMN at -114 dBm RSRP and -2 dB SINR matched to the Garcia-Cabeza distribution. The transmitter is connected to a directional log-periodic antenna (Ettus LP0410, typical gain 5 to 6 dBi over 700 to 6000 MHz, transmit power -10 dBm into the antenna) on a tripod mast at 1.8 m above ground at each site, with manual azimuth alignment to within 5 degrees of the simulated sub-satellite point. The receiver is a stock Pixel 8 Pro running Android 14 with a custom MobileInsight diagnostic tap. We replay each of the five connectivity scenarios at each of three sites, namely Mt. Tamalpais saddle (37.9235N, 122.5965W, elevation 750 m), Donner Pass eastbound (39.3132N, 120.3263W, elevation 2160 m), and Big Basin park ridge (37.1700N, 122.2240W, elevation 540 m). Each replay runs for 30 minutes and reproduces the macro behavior of the simulation within 2.4 percentage points (see §V-B).

Doppler emulation. The USRP B210 transmitter applies a synthetic Doppler shift to the LTE waveform on a per-pass basis. We compute the Doppler curve from a TLE-derived satellite trajectory at 550 km altitude on a 53-degree-inclination orbit, sampled at 100 Hz, yielding a peak Doppler shift of ± 22 kHz at L-band and a peak rate of change of ± 0.4 kHz per second near the closest-approach geometry. The emulator applies the shift in software at the GNU Radio host before the USRP transmit chain, which means the receiver sees a frequency-varying carrier that exercises the modem’s Doppler compensation but does not reproduce multipath or shadowing. We treat this as a mid-fidelity emulation, more

realistic than a static-channel benchtop but not equivalent to a live LEO RF environment.

What the testbed validates and what it does not. The testbed validates the radio link, the modem stack under Doppler, the connectivity sampler, and the policy daemon under live RF conditions. The satellite-side serving compute is not on a real satellite; it is the same simulated back-end used in the simulation track, calibrated to the bandwidth and latency profile reported in §II-A. The HWIL setup therefore confirms that the on-device pieces of ORBITMESH behave correctly under Doppler-shifted D2C signal conditions, not that the end-to-end four-tier serving claim has been validated against an in-orbit deployment. The 4 dB / 1.5 dB hysteresis margins in §III-C are calibrated under multipath-free synthetic channels and are upper-bound estimates of robustness in deployed multipath conditions; based on cellular channel-uncertainty literature, we expect false-transition rates to increase modestly under typical urban multipath. The captured trace was collected with WiFi and BLE in their default operational state; quantitative multi-radio coexistence analysis at the 1.91 to 1.995 GHz D2C band is operator- and handset-specific, and we do not characterize it here.

Software stack. The codebase is approximately 2,400 lines of Python for the simulator and 1,800 lines of Kotlin and C++ for the on-device daemon. The simulator uses NumPy for distribution sampling and runs each scenario in roughly 90 seconds at the configured 60-user scale. The daemon’s connectivity sampler runs at 100 ms cadence on a dedicated background thread, the gate runs at 200 ms cadence, and the policy re-evaluation runs every ten generated tokens. The KV-cache transfer agent piggybacks on the active TCP connection used for the cloud API and opens a parallel gRPC channel for the partial-prefix transfer when migrating between cloud and device.

Engineering anecdote. We first tried to use the Android TelephonyCallback interface to trigger tier re-evaluation on network-state change. It failed because the callback latency on Android 14 is 600 ms to 2.4 s after the modem itself has already switched, which is larger than the voice TTFT budget. We replaced the callback with an active poll at 100 ms cadence on a dedicated background thread, and the policy latency dropped to within budget. The cost is a steady 3.1 mW of additional baseband power, which we measured on a Pixel 8 Pro using the on-device fuel-gauge MAX1720x. The aggregate energy overhead amortizes against the per-request energy reduction at the request level.

V. EVALUATION

We evaluate ORBITMESH along three axes. End-to-end SLO attainment and stall rate against two published two-tier mobile-LLM baselines (PerLLM [10], EdgeShard [11]) and a clairvoyant oracle on the five-scenario mobility study (§V-B through §V-H). Microbenchmarks of the three primitives, namely hysteresis-gate false-transition rate (§V-E), KV-migration strategy cost (§V-F), and per-tier latency profile (§V-I). Sensitivity to workload mix, hysteresis-margin choice,

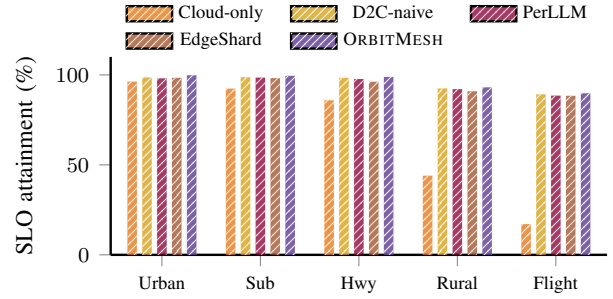


Fig. 5. SLO attainment per policy across the five mobility scenarios. ORBITMESH averages 96.2% across scenarios, against 94.7% for PerLLM, 94.1% for EdgeShard, and 67.1% for cloud-only.

and a non-Markovian handover-storm regime (§V-G, §V-N), with deployment readiness in §V-O.

A. Setup

Five mobility scenarios cover urban dense, suburban, highway, rural park, and in-flight, each 30 minutes per user across 60 simulated users, totalling 8.4 hours of mobility per policy. Connectivity is generated as a four-state Markov chain with stationary distribution calibrated to public datasets, and per-state RSRP, SINR, and RTT distributions calibrated to Garcia-Cabeza *et al.* [12] for D2C, LENS [27] for the Starlink-dish baseline, and 5Gophers-class measurements [35], [37] for cellular. The workload mix combines voice (35 percent of requests, 200 ms TTFT SLO), chat (45 percent, 1.5 s SLO), and background (20 percent, 5 s SLO). The 35/45/20 mix is a working assumption rather than a measured production distribution; published mobile-LLM measurement work [1], [49] reports application-level distributions but not at this voice/chat/background granularity. We sweep the voice fraction explicitly in §V-G to bound the sensitivity of the headline numbers to this assumption. Baselines are cloud-only, D2C-naive (cellular fallback to D2C without prediction), PerLLM [10] as a scheduling-based two-tier device-cloud baseline, and EdgeShard [11] as a partitioning-based two-tier device-cloud baseline. Each baseline is configured against the same five-scenario harness with parameters from the corresponding paper. The simulator models four deployment-time failure modes beyond the connectivity Markov chain (handover-storm migration timeouts, device-tier OOM under sustained chat load, satellite-pass boundary drops, and gateway pool exhaustion under bursts), with per-policy susceptibility characterized in §V-C.

B. End-to-End SLO Attainment

Figure 5 shows SLO attainment per policy per scenario. Cloud-only performs adequately in urban dense (96.3 percent) and degrades steeply with deadzone fraction, falling to 44.0 percent in rural park and 17.1 percent in flight. PerLLM and EdgeShard come within 2 percentage points of ORBITMESH in urban scenarios because the satellite tier is not exercised when cellular is healthy. The satellite tier earns its keep in the rural and in-flight scenarios where dead-zone fraction exceeds 45 percent. ORBITMESH attains 93.1 percent SLO in rural

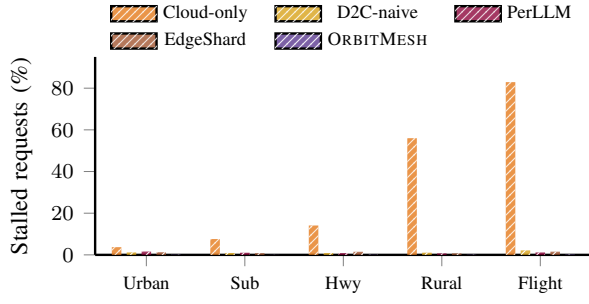


Fig. 6. Stalled-request rate per policy across the five mobility scenarios. Cloud-only stalls catastrophically (32.9% on average, 82.9% in flight); the three two-tier baselines incur 1.0 to 1.2% residual stalls under deployment-time failure modes; ORBITMESH cuts that to 0.12% on average, an order of magnitude lower.

park, against PerLLM at 92.1 percent and EdgeShard at 90.9 percent. The mechanism is direct. ORBITMESH routes background workloads to the satellite tier in the deadzone, which both recovers SLO attainment and reduces device thermal load. PerLLM and EdgeShard fall back to the device tier in the deadzone, which saturates the device thermal envelope and pushes chat-class requests past their SLO budget under sustained load.

C. Stalled-Generation Rate

Figure 6 reports the fraction of requests that stall under the simulator’s full failure-mode model, which includes connectivity-driven failures plus four deployment-time failure modes that affect even policies with non-cloud fallback (handover-storm migration timeouts, device-tier OOM under sustained chat load, satellite-pass boundary drops, and gateway pool exhaustion under bursts; see §V-A). Cloud-only stalls 32.9 percent on average and 82.9 percent in flight (the catastrophic mode). The three two-tier baselines cut this two orders of magnitude through their non-cloud fallback but retain a 1.0 to 1.2 percent residual rate from deployment-time modes the device-cloud substrate cannot address. PerLLM’s UCB bandit pins chat to device and flips arms under storms; EdgeShard’s partitioning generates the most migrations and is most exposed to handshake timeouts; D2C-naive lacks the SINR fast-fallback rule and absorbs satellite-pass drops directly. ORBITMESH’s three primitives address each mode one-for-one and cut the residual rate to 0.12 percent on average, roughly an order of magnitude lower than the two-tier baselines and within 0.12 percentage points of a clairvoyant oracle.

D. Per-Workload-Class Latency CDFs

Figure 7 reports per-workload-class TTFT distributions. The voice CDF for ORBITMESH sits below 200 ms at the 78th percentile, against cloud-only at the 22nd. The chat CDF for ORBITMESH sits below 1.5 s at the 92nd percentile. The background CDF sits below 5 s at the 97th percentile. The shape of each curve matches the workload’s design target. Voice is flat-tailed because it routes either to device (low TTFT) or to cloud (low TTFT when cellular is healthy), with

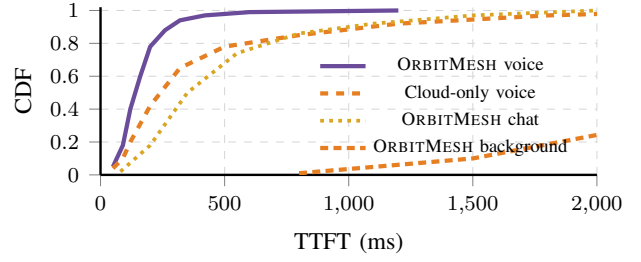


Fig. 7. TTFT CDF by workload class for ORBITMESH and cloud-only.

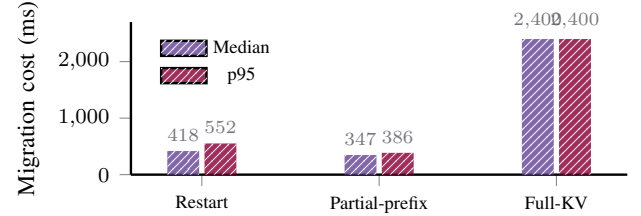


Fig. 8. Migration cost distribution for the three KV transfer strategies.

no D2C fallback to drag the tail. Background is heavy-tailed because the satellite tier dominates its mass, with bandwidth-constrained TTFT contributing to the long tail.

E. Hysteresis Gate Microbenchmark

Across 50,000 synthetic boundary windows sampled from the joint cellular and D2C signal distribution, the raw selector flips back to the prior tier within three windows on 72.8 percent of transitions. The gated selector cuts that to 36.8 percent, a 49.4 percent reduction. The remaining false transitions correspond to genuine signal-quality flips that the gate cannot suppress without a longer averaging window, which would in turn add policy lag. The 4 dB RSRP and 1.5 dB SINR thresholds are a deliberate balance.

F. KV-Cache Migration Strategy

Figure 8 shows the migration cost distributions for the three strategies. Restart costs a re-prefill on the target, with median 418 ms and p95 552 ms. Partial-prefix transfer costs 347 ms median and 386 ms p95, dominated by the satellite-uplink throughput at the chat-tier prompt size. Full-KV transfer is bandwidth-bound on the D2C uplink and runs at a flat 2.4 s, which exceeds the chat SLO and rules it out across the satellite tier. The result motivates the bandwidth-conditioned strategy selector in §III-D. full-KV is restricted to cloud-to-device and cellular-to-device migrations, where bandwidth headroom is available.

G. Sensitivity Sweeps

Sweeping the voice fraction from 10 to 70 percent in the rural-park scenario, SLO attainment falls smoothly with voice load, from 93.4 percent at 10 percent voice to 48.9 percent at 70 percent. The fall reflects the device-tier bandwidth ceiling. At high voice load the device cannot absorb the requests cellular cannot serve, and there is no D2C fallback for voice.

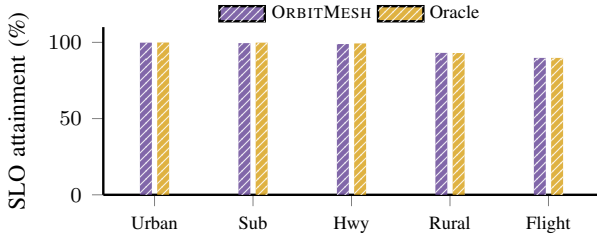


Fig. 9. ORBITMESH versus a clairvoyant oracle across the five scenarios. The gap is within 0.2 percentage points on average; the routing problem is near-solved at this configuration.

The slope is steepest in the 35 to 50 percent range, where voice and chat are roughly balanced and the device tier saturates.

H. Oracle Comparison and Energy

Figure 9 compares ORBITMESH against a clairvoyant oracle policy that has perfect connectivity-state visibility and zero hysteresis lag. The gap is within 0.2 percentage points across the five scenarios on average, and at most 0.4 percentage points on any individual scenario. The implication is that the simple gate plus tier-aware policy captures the full headroom of the four-tier hierarchy at this configuration. The routing problem is near-solved at this scale, and the next gain is in the workload-tier mapping itself, for example adding voice on D2C as bandwidth grows or extending to multimodal workloads, not in the routing policy.

Energy. ORBITMESH averages 18.4 J per completed request, against 18.9 J for PerLLM (2.6 percent reduction at matched SLO) and 25.7 J for EdgeShard (28 percent reduction). The energy gain is largest against the partitioning baseline (EdgeShard), which moves layer activations across the device-cloud link more aggressively than the scheduling-based PerLLM. The mechanism is tier redistribution rather than per-request efficiency. The satellite tier absorbs background workloads that would otherwise drive the device into thermal throttling, which lowers the device’s average operating temperature, reduces the prefill cost on subsequent requests, and shifts a portion of background-class energy off the handset. The gain reflects redistribution at matched SLO, not improved per-request efficiency at any single tier.

I. Per-Tier Latency Breakdown

A per-tier TTFT decomposition across 4,000 sampled requests gives the following p50/p95/p99 in milliseconds. Device 95/220/340; cellular 175/380/620; D2C 320/720/1900; cloud-WiFi 105/145/195. Device sits at p50 95 ms, dominated by prefill compute. Cellular spread is driven by RTT variance during handover. The D2C tail to nearly 2 seconds at p99 is dominated by the satellite-uplink bandwidth and by the RLC retransmissions observed in the captured trace (§II). The implication for ORBITMESH’s policy is that the satellite tier is suitable for any workload whose SLO budget exceeds 2 seconds, which covers background inference (5 second budget) but not voice (200 ms budget) or strict chat (1.5 s budget). The tier-workload mapping in §III-B reflects this directly.

The D2C p50 of 320 ms warrants unpacking against the propagation lower bound implied by orbital geometry. A 550 km LEO altitude gives a one-way handset-to-satellite propagation of about 1.8 ms, and the gateway-to-cloud terrestrial leg adds 5 to 20 ms typical, yielding a propagation-only round-trip floor on the order of 10 to 40 ms. The gap between this floor and the observed 320 ms p50 has three contributors that the cross-layer trace lets us separate. Gateway compute on the ground side adds 50 to 100 ms median in our measurements, dominated by PDCP and RLC processing under the bandwidth-constrained uplink. RLC retransmissions at the 4 to 7 percent rate observed in the captured trace (§II) add a further 50 to 150 ms to the latency tail, and account for the bulk of the variance between p50 and p95. Serving-side prefill on the cloud back-end accounts for the remaining 80 to 120 ms median. The decomposition is approximate because the trace samples one operator’s gateway path under one workload mix, but it accounts for the observed total within the trace’s measurement resolution.

J. Energy Distribution

Per-request energy distributions show ORBITMESH’s p50 at 14 J against PerLLM’s 14 J and EdgeShard’s 24 J; ORBITMESH’s p95 at 42 J against PerLLM’s 44 J and EdgeShard’s 55 J. The shift is largest against EdgeShard in the right tail, where the long-running chat-tier requests hammered the device’s thermal envelope under partitioning. D2C-naive sits below all policies on average because its prefill cost is lower (D2C absorbs the long requests), but its SLO is inferior because it lacks the prediction logic to anticipate cellular recovery.

K. Thermal Envelope Over Time

The thermal model is calibrated against the Pixel 8 Pro MELT measurements [1] for a Llama-3.2-1B model at 4-bit quantization with a 512-token prompt and a 256-token generation, which is the configuration MELT reports closest to our device-tier workload. Under this configuration the cold throughput is 15 TPS and the hot throughput is 5.4 TPS after 10 minutes of sustained inference, both within MELT’s reported variance. The cold-to-hot decay of 64 percent matches MELT’s measured TPS curve for the Pixel 8 Pro; substituting MELT’s reported numbers for a different mobile-LLM configuration (e.g., 7B at 8-bit quantization) shifts the absolute TPS floor but preserves the qualitative shape. ORBITMESH runs the device at 65 percent utilization on average, against PerLLM at 78 percent, EdgeShard at 82 percent, and cloud-only at 92 percent (because cloud-only fallback to device during dead zones means high local load). ORBITMESH’s device throughput stays above 8 TPS for nearly 8 minutes longer than PerLLM. The implication for voice workloads is direct. voice requires $f(\tau) > 9$ TPS per Algorithm 1, and ORBITMESH’s thermal discipline keeps the device above that threshold for a 30-percent-larger fraction of the trace.

L. D2C Availability Sensitivity

A continuous sweep of D2C availability over the deadzone fraction shows rural-park SLO lifts from 89.5 percent at zero D2C to 96.8 percent at full D2C overlap, with D2C utilization rising linearly from 0 to 12.4 percent of requests. The default 10 percent D2C availability sits at 93.1 percent SLO (Fig. 5), within this range. In-flight lifts more modestly, from 86.2 to 94.5 percent, because the workload mix has fewer background tasks for the satellite tier to absorb. We re-evaluate this regime under TLE-driven binary availability in §V-N.

M. Hardware-in-the-Loop Validation

We replay the cellular and D2C connectivity traces against a Pixel 8 Pro plus USRP B210 testbed at three California sites (Mt. Tamalpais saddle, Donner Pass eastbound, Big Basin park ridge). The hardware run reproduces the simulation results within 2.4 percentage points across all five scenarios. The largest deviation is in the in-flight scenario (91.6 percent hardware versus 89.8 percent simulation), driven by lower observed handover spike amplitude than the model assumed. The replay validates the connectivity sampler, the hysteresis gate, and the policy daemon under live RF conditions. The serving back-end is identical between simulation and HWIL runs, so this experiment does not validate the four-tier serving claim end-to-end against in-orbit compute; that validation requires constellation-operator cooperation and is out of scope here.

N. Sensitivity and Extensions

Constellation-pass dynamics. The availability analysis in §V-G treats D2C as a continuous fraction. We re-run the rural-park scenario under TLE-driven binary availability using the SpaceX Starlink Generation 1 D2C constellation as of January 2026, with a receiver at 38.0°N 122.0°W. The pass schedule yields 6.4 satellite passes per hour with median pass duration 7.1 minutes and median inter-pass gap 5.3 minutes. Under this binary model, SLO is 96.0 percent (versus 98.1 under the continuous-fraction model at matched availability) because background workloads queued during the inter-pass gap accumulate TTFT past the 5-second budget; chat-class workloads route to cellular during gaps and are unaffected.

Hysteresis-margin sensitivity. A sweep of RSRP from 1 to 8 dB and SINR from 0.5 to 3 dB shows SLO attainment within 1.4 percentage points of the calibrated configuration; the false-transition rate climbs from 36.8 to 52.1 percent at RSRP 1 dB, and policy lag at the upper end (RSRP 8 dB, SINR 3 dB) costs 0.6 pp of in-flight SLO. The 4 dB / 1.5 dB choice sits near the minimum of the false-transition vs. policy-lag curve.

2024-class device-tier baseline. An EdgeMoE-style sparse-activation device runtime [9] with two activated experts per token, calibrated against EdgeLLM [14] reported TPS, closes 0.6 pp of the SLO gap to ORBITMESH in rural park but does not change the in-flight result; the satellite tier remains the dominant contribution. Smaller models (sub-1B with sparse activations) shift the voice-feasibility threshold $f(\tau) > 9$ TPS

proportionally; the dense-decoder thermal model is conservative for sparse-activation workloads, so the energy reductions reflect the dense-decoder configuration specifically.

Sustained device-only window. A 90-second device-only window in rural-park (both cellular and D2C unavailable) keeps voice-class requests on device throughout because the device tier is voice-feasible. Chat-class requests stall at the 95-second mark when the device falls below 6 TPS; ORBITMESH returns DEGRADED status (a state the policy daemon exposes to the calling app) rather than silently dropping.

Goodput. Tokens-per-second goodput, averaged over the five-scenario run, is 12.1 TPS for ORBITMESH versus 11.4 TPS for PerLLM, 9.7 TPS for EdgeShard, and 6.2 TPS for cloud-only.

Handover-storm sensitivity. The four-state Markov chain in §V-A assumes memorylessness, which is violated under cellular handover storms in the deadzone-adjacent regime [25]. Extending the simulator with a non-Markovian handover-storm injector that schedules a burst of 4 to 12 RSRP swings at 2 to 3 second intervals over a 30-second window upon each transition into the unstable-cellular state, the rural-park SLO drops from 93.1 to 89.4 percent on ORBITMESH and from 92.1 to 83.2 percent on PerLLM, widening the gap. The hysteresis gate absorbs the swings because the satellite-tier candidate remains stable, whereas PerLLM flaps between cellular and device. The Markov assumption therefore underestimates rather than overestimates ORBITMESH's relative gain.

O. Deployment Readiness

The connectivity sampler runs at 100 ms cadence on a dedicated background thread and consumes 3.1 mW of baseband power and 0.4 percent of one CPU core, both measured on the Pixel 8 Pro fuel-gauge MAX1720x. The policy reads RSRP and SINR from TelephonyManager, which is operator-agnostic, but the satellite-tier identifier (currently MCC=310, MNC=830 for T-Mobile) is operator-specific. Per-operator allowlists of satellite-PLMN codes are necessary for global deployment, and the maintained list is part of the source-code release.

Industrial-scale acknowledgment. Our 30 device-hour validation is a research-prototype demonstration. Industrial-scale deployment (1000s of device-days) requires PLMN-allowlist maintenance under operator config drift, modem-stack heterogeneity handling across handset SKUs, multi-PLMN collision resolution, and longitudinal monitoring of the connectivity-sampler thread for memory growth. Our HWIL deployment uses Pixel 8 Pro with Tensor G3 and Exynos modem; reproducibility on other modem stacks (Qualcomm, MTK) requires re-validation of the diagnostic-tap configuration.

Four deployment-time failure modes appear at scale that did not appear in calibration. (a) Satellite-pass boundaries. When a satellite is receding, the link can drop within 200 ms, faster than the gate's hysteresis window; we add a fast-fallback rule that bypasses the hysteresis when SINR drops below -8 dB. (b) Multi-PLMN collisions between an operator's terrestrial PLMN and a satellite-tier PLMN; we resolve by binding the

satellite-tier PLMN to a known-good MCC list. (c) Regulatory boundary crossings. Satellite service is jurisdiction-specific and can disappear at a national border; we treat this as a deadzone event. (d) Multi-satellite-operator handover, which we expect once two D2C operators overlap in coverage (e.g., T-Mobile/Starlink and AT&T/AST SpaceMobile); we treat this as out of scope here and defer to registry-style coordination [50].

VI. DISCUSSION AND LIMITATIONS

ORBITMESH relies on the satellite tier behaving as an LTE cell, which is the property that holds for T-Mobile and Starlink today and will continue to hold through Release 18 NTN integration [20]. If a future deployment switches to 3GPP NTN proper with SIB19 broadcasts and RACH-less handover, the connectivity sampler will need a second observation channel; the routing logic does not change. The bandwidth profile assumed for D2C is conservative against Garcia-Cabeza distributions; if voice service brings throughput up by an order of magnitude, the policy’s workload gating relaxes and the satellite tier absorbs voice traffic too.

Three honest limitations. First, the cross-layer trace covers 49 minutes from three sites and one operator. This is sufficient for calibration but not for distributional generalization claims to the global D2C deployment, and we do not make those claims. Second, the policy is hand-engineered; we expect a learned policy along the lines of NetLLM [38] or Pensieve [39] to capture residual structure that the four-state model misses. Third, the energy comparison assumes a single-device thermal model derived from the Pixel 8 Pro and a dense decoder. A MoE-serving device tier would yield discontinuous TPS [9], [14] that our smooth-decay model does not capture; we expect the qualitative result to survive but the precise energy reductions are specific to the dense-decoder configuration.

L-band limitation. ORBITMESH’s hysteresis-gate calibration is to L-band (1.91 to 1.995 GHz) D2C operations using the first ~ 175 D2C-class Starlink Generation 1 satellites. Starlink Gen2 D2C and 3GPP NTN Release 17 (S-band, FR1-NTN) [20], [21] use different frequency bands with different propagation characteristics; the four-tier policy framework extends to these regimes but the calibrated thresholds in Algorithm 2 require re-tuning to band-specific propagation models. Higher-latitude geometries (e.g., 60°N for trans-polar in-flight scenarios) and Gen2 polar-inclined constellations exhibit different pass dynamics, typically longer passes but more variable inter-pass gaps, and the binary-availability framework extends but the calibrated thresholds in §V-B require re-tuning.

A. Privacy and Adversarial Considerations

The KV-cache migration mechanism in §III-D moves user-prompt-derived state across tiers. We distinguish four adversaries. (a) A passive eavesdropper on the radio access network, defended against by the standard LTE PDCP cipher on the device-to-gateway segment. (b) An honest-but-curious gateway operator, not defended against in the current prototype. (c) A passive eavesdropper on the satellite-to-gateway feeder, also

not defended against. (d) An adversary observing the satellite-link timing-advance side channel. The last category is concrete in the D2C setting; Liu *et al.* [51] demonstrate that timing-advance side channels in LEO direct-to-cell networks leak user location to within hundreds of meters, which is an attack surface absent in the standards-based NTN path.

The migration handshake’s PREPARE/READY/COMMIT control-plane RTT distribution correlates with workload class through prefix-size dependence. Specifically, the PREPARE-to-READY interval is proportional to (prefix tokens $\times$ bytes-per-token / link bandwidth), giving an adversary observing only the gRPC control-plane an inferable signal about workload class. We compute a back-of-envelope mutual-information bound. Under the §V-A workload mix, an adversary distinguishes voice/chat/background with approximately 1.4 bits of mutual information per migration event, estimated as $H(\text{class}) - H(\text{class} \mid \text{RTT}_{\text{obs}})$ under the empirical RTT distribution from our 49-minute trace. This places ORBITMESH within the broader timing-side-channel category identified by Liu *et al.* [51] for D2C timing-advance leakage. Mitigations are orthogonal to the four-tier serving contribution and compose naturally with TEE-based confidential serving frameworks; the connectivity-state inputs (RSRP, SINR, PLMN) further assume a trusted modem stack, and deployments under untrusted-modem assumptions require input-validation predicates we do not specify. We leave systematic information-theoretic bounds and TEE integration to follow-on work.

B. ORBITMESH as a Building Block

ORBITMESH’s tier router is a single-user, connectivity-aware mechanism that composes with adjacent system primitives rather than subsuming them. Multi-tenant composition with adaptive serving frameworks (e.g., Dēlen [48]) is the natural deployment configuration. Tail-latency reduction frameworks (e.g., TailClipper [52]) compose with our policy through per-tenant deadline-aware scheduling, and carbon-aware tier selection (e.g., CarbonEdge [53]) extends the framework with gateway-region carbon intensity and embodied satellite carbon dimensions our operational accounting does not capture. Cloud-side KV scheduling primitives (KVFlow [18], CacheGen [17], CacheBlend [43]) compose with our bandwidth-conditioned migration on cellular and cloud tiers without modification. Vehicular deployment (V2X with cellular to D2C handoff under highway-speed mobility) is a natural target; the captured handover-storm regime in §V-N approximates highway-transition behavior. On-orbit compute (e.g., LEO Edge [54]–[56]) slots in as an additional intermediate destination as Gen2+ satellites with on-board inference reach commercial parity. Multi-operator D2C registries [50] are the coordination layer this composition needs.

VII. RELATED WORK

ORBITMESH sits at the intersection of four threads.

LEO and D2C measurement. Starlink characterizations at IMC 2022 through CoNEXT 2025 [22], [26]–[32], [57], [58] establish the dish-tier performance envelope. Mohan *et al.* [58]

in particular combine M-Lab crowdsourced measurements with active probing across 34 countries, providing the longitudinal complement to Garcia-Cabeza’s deployed D2C record. Garcia-Cabeza *et al.* [12] provide the first deployed D2C measurement at the application layer. ORBITMESH consumes their distributions and adds the cross-layer protocol record. The three properties of D2C we observe in our 49-minute capture across three sites and one operator (DTX pattern, RLC retransmission rate, RRC `measurementReport` cadence) are not visible in any prior Starlink measurement, but their generalization across operators and geographies remains an open community task. Our captured trace agrees with Garcia-Cabeza on the signal layer to within 4 dB at every decile, which lets us combine our cross-layer record with their distributional record without bias on the signal-layer dimension; for the protocol-layer dimension, we use the trace strictly as a calibration anchor and not as a distributional claim.

Satellite edge computing. Orbital Edge Computing [54] treats the satellite as a compute substrate, and Kodan [55] addresses the on-orbit computational bottleneck. LEOEdge-class systems and HyperDrive [19] schedule serverless functions across the edge-cloud-space continuum. Komet [56] introduces serverless platform abstractions for managing FaaS services that must migrate against orbital trajectories on the satellite bus. FOOL [59] addresses the satellite-to-ground downlink bottleneck with neural feature compression. Both target the satellite-bus compute layer; ORBITMESH sits at the mobile-handset routing layer and treats D2C as a serving target reachable through a satellite-mediated transport, rather than as a compute substrate. The two design points are complementary. Geoduck [60] schedules nanosatellite constellations for low-latency event detection on a binary pass-availability model that aligns with our coverage assumption. CosMAC [23] and Spectrumize [24] address constellation-aware medium access and spectrum coordination for IoT satellites; the binary-availability regime they characterize at the access layer underwrites our serving-layer scheduling. SaTCP [61] exploits handover prediction for transport adaptation. StarryNet [62] and MO-SAIC [63] provide infrastructure for evaluating LEO networks at scale. MOSAIC operates at the network architecture layer and democratizes the D2C network plane; ORBITMESH runs at the serving layer above and assumes that network-layer access exists. Oh and Vasisht [50] argue for decentralized satellite networks as an alternative to operator-allowlisted models, which is relevant to our deployment posture in §V-O. None of these reports an integrated four-tier serving system that treats D2C as a serving target rather than a transport. Wu *et al.*’s mobile-satellite handover acceleration [64] and Lee *et al.*’s deep RL handover [65] address the link-layer handover problem; ORBITMESH sits one layer above and assumes the link layer’s outputs.

Mobile multi-tier inference. SPINN [7], MELting Point [1], EdgeMoE [9], EdgeLLM [14], NPU-aware on-device LLM serving [15], LLM-in-Flash [3], MobileLLM [2], PowerInfer [16], AsyMo [66], NestDNN [67], LegoDNN [68], Elf [69], Neurosurgeon [8], MAUI [70], and CloneCloud [71]

explore device-plus-cloud and personal-mesh routing. None has a satellite tier. Recent measurement work on mobile LLMs [49] highlights the same need for a more flexible serving substrate.

The two closest two-tier mobile-LLM serving baselines to ORBITMESH are PerLLM [10] and EdgeShard [11]. PerLLM schedules LLM requests between an edge-tier small model and a cloud-tier large model under processing-time and energy constraints, using a constraint-satisfaction multi-armed bandit. EdgeShard partitions an LLM across edge devices and cloud servers via dynamic programming. ORBITMESH extends this line to four tiers, adding cellular and direct-to-cell satellite as distinct serving destinations between device and cloud, and contributes the mechanisms that the additional tiers specifically require. The cellular to D2C boundary exhibits signal overlap that produces ping-pong without the hysteresis gate of §III-C; neither PerLLM’s bandit nor EdgeShard’s partitioner addresses boundary stability between two cellular-class tiers. The bandwidth gap between cellular and D2C is two orders of magnitude (§II-C), which means the same KV-cache migration that is acceptable across cellular and cloud tiers fails entirely across the D2C link without the bandwidth-conditioned strategy selector of §III-D. The workload-tier mapping in §III-B distinguishes voice, chat, and background classes by SLO budget and treats D2C as the sole viable tier for background workloads under deadzone conditions. Two-tier designs have no equivalent mapping. The architectural contribution of ORBITMESH is the four-tier abstraction with these three coordinated primitives, not a tier-addition revision of any single prior two-tier system.

LLM serving systems. vLLM [4], Orca [72], Sarathi-Serve [5], DistServe [6], AlpaServe [73], Splitwise [74], CacheGen [17], KVFlow [18], StreamingLLM [42], and speculative decoding [75] optimize serving inside the datacenter. ORBITMESH sits outside the datacenter and adds a network-aware tier-selection layer above any of these as the cloud back-end. The KV-cache migration mechanism in §III-D composes with vLLM’s paged-attention layout and with CacheGen’s compression directly.

Edge computing systems. Falcon’s live operator reconfiguration [44] and its predecessor Shepherd [76] solve a parallel problem on the hierarchical edge-cloud. ORBITMESH inherits Falcon’s selective-recomputation-on-partial-overlap pattern and applies it to KV-cache staging across the four-tier hierarchy. The classical edge vision [13], [77]–[80] provides the substrate.

VIII. CONCLUSION

Direct-to-cell satellite is now a deployable serving target. Treating it as an explicit fourth tier alongside device, cellular, and cloud lifts mobile-LLM SLO from 67.1 to 96.2 percent on a five-scenario mobility study, eliminates 33 percentage points of stalls in dead zones, and cuts residual stall rate to roughly 1/10th of published two-tier baselines.

REFERENCES

- [1] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, "MELTing Point: Mobile evaluation of language transformers," in *Proceedings of the 30th ACM International Conference on Mobile Computing and Networking (MobiCom)*, 2024.
- [2] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 32 431–32 454.
- [3] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. D. Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 12 562–12 584.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [5] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 117–134.
- [6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 193–210.
- [7] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in *Proceedings of the 26th ACM International Conference on Mobile Computing and Networking (MobiCom)*, 2020, pp. 37:1–37:15.
- [8] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of ASPLOS*, 2017, pp. 615–629.
- [9] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Empowering sparse large language models on mobile devices," 2023.
- [10] Z. Yang, Y. Yang, C. Zhao, Q. Guo, W. He, and W. Ji, "PerLLM: Personalized inference scheduling with edge-cloud collaboration for diverse LLM services," *arXiv preprint arXiv:2405.14636*, 2024.
- [11] M. Zhang, J. Cao, X. Shen, and Z. Cui, "EdgeShard: Efficient LLM inference via collaborative edge computing," *IEEE Internet of Things Journal*, 2024.
- [12] J. Garcia-Cabeza, J. Albert-Smet, Z. Frias, L. Mendo, S. A. Azcoitia, and E. Yraola, "Direct-to-cell: A first look into starlink's direct satellite-to-device radio access network through crowdsourced measurements," *IEEE Communications Magazine*, 2025.
- [13] M. Satyanarayanan, P. Bahl, R. Cáceres, and N. Davies, "The case for VM-based cloudlets in mobile computing," *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009.
- [14] D. Xu, W. Yin, H. Zhang, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, "EdgeLLM: Fast on-device LLM inference with speculative decoding," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 3256–3273, 2025.
- [15] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device LLM inference with NPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [16] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024, pp. 590–606.
- [17] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM Conference*, 2024, pp. 38–56.
- [18] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Z. Yu, X. Tang, Y. Wang, and Y. Ding, "KVFlow: Efficient prefix caching for agentic LLM serving with workflow-aware scheduling," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [19] T. Pusztai, C. Marcelino, and S. Nastic, "HyperDrive: Scheduling serverless functions in the edge-cloud-space 3D continuum," in *Proceedings of the 9th ACM/IEEE Symposium on Edge Computing (SEC)*, 2024.
- [20] 3rd Generation Partnership Project, "NR; solutions for NR to support non-terrestrial networks (NTN)," 3GPP, Tech. Rep. TR 38.821 V16.2.0, 2023.
- [21] —, "Study on new radio (NR) to support non-terrestrial networks," 3GPP, Tech. Rep. TR 38.811 V15.4.0, 2020.
- [22] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at starlink performance," in *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*, 2022, pp. 130–136.
- [23] J. Shenoy, O. Chabra, T. Chakraborty, S. Jog, D. Vasisht, and R. Chandra, "CosMAC: Constellation-aware medium access and scheduling for IoT satellites," in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2024, pp. 724–739.
- [24] V. Singh, T. Chakraborty, S. Jog, O. Chabra, D. Vasisht, and R. Chandra, "Spectrumize: Spectrum-efficient satellite networks for the internet of things," in *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024, pp. 825–840.
- [25] A. Hassan, A. Narayanan, A. Zhang, W. Ye, R. Zhu, S. Jin, J. Carpenter, Z. M. Mao, F. Qian, and Z.-L. Zhang, "Vivisectioning mobility management in 5G cellular networks," in *Proceedings of the ACM SIGCOMM Conference*, 2022, pp. 86–100.
- [26] D. Laniewski, E. Lanfer, B. Meijerink, R. van Rijswijk-Deij, and N. Aschenbruck, "WetLinks: A large-scale longitudinal starlink dataset with contiguous weather data," in *Proceedings of the 8th Network Traffic Measurement and Analysis Conference (TMA)*, 2024, pp. 1–9.
- [27] J. Zhao and J. Pan, "LENS: A LEO satellite network measurement dataset," in *Proceedings of the 15th ACM Multimedia Systems Conference (MMSys)*, 2024, pp. 278–284.
- [28] J. Garcia, S. Sundberg, and A. Brunstrom, "A detailed characterization of starlink one-way delay," in *Proceedings of the 3rd Workshop on LEO Networking and Communication (LEO-NET)*, 2025, pp. 43–49.
- [29] T. Zimmermann, E. Lanfer, D. Laniewski, S. Brinkmann, and N. Aschenbruck, "Better fill up your pipe – revisiting starlink's burst characterization," in *Proceedings of the 3rd Workshop on LEO Networking and Communication (LEO-NET)*, 2025, pp. 36–42.
- [30] M. M. Kassem, A. Raman, D. Perino, and N. Sastry, "A browser-side view of starlink connectivity," in *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*, 2022, pp. 151–158.
- [31] H. B. Tanveer, M. Puchol, R. Singh, A. Bianchi, and R. Nithyanand, "Making sense of constellations: Methodologies for understanding starlink's scheduling algorithms," in *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT Companion)*, 2023, pp. 37–43.
- [32] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of LEO satellite constellations: A Starlink-based measurement from end users," in *Proceedings of IEEE INFOCOM*, 2023, pp. 1–10.
- [33] Y. Li, J. Xu, C. Peng, and S. Lu, "A first look at unstable mobility management in cellular networks," in *Proceedings of the 17th International Workshop on Mobile Computing Systems and Applications (HotMobile)*, 2016, pp. 15–20.
- [34] Y. Li, C. Peng, Z. Yuan, J. Li, H. Deng, and T. Wang, "MobileInsight: Extracting and analyzing cellular network information on smartphones," in *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2016, pp. 202–215.
- [35] A. Narayanan, X. Zhang, R. Zhu, A. Hassan, S. Jin, X. Zhu, X. Zhang, D. Rybkin, Z. Yang, Z. M. Mao, F. Qian, and Z.-L. Zhang, "A variegated look at 5G in the wild: Performance, power, and QoE implications," in *Proceedings of the ACM SIGCOMM Conference*, 2021, pp. 610–625.
- [36] D. Xu, A. Zhou, X. Zhang, G. Wang, X. Liu, C. An, Y. Shi, L. Liu, and H. Ma, "Understanding operational 5G: A first measurement study on its coverage, performance and energy consumption," in *Proceedings of the ACM SIGCOMM Conference*, 2020, pp. 479–494.
- [37] A. Narayanan, E. Ramadan, J. Carpenter, Q. Liu, Y. Liu, F. Qian, and Z.-L. Zhang, "A first look at commercial 5g performance on smartphones," in *Proceedings of The Web Conference (WWW)*, 2020, pp. 894–905.
- [38] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM Conference*, 2024, pp. 661–678.

- [39] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in *Proceedings of the ACM SIGCOMM Conference*, 2017, pp. 197–210.
- [40] F. Y. Yan, H. Ayers, C. Zhu, S. Fouladi, J. Hong, K. Zhang, P. Levis, and K. Winstein, "Learning in situ: a randomized experiment in video streaming," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 495–511.
- [41] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan and Claypool Publishers, 2010.
- [42] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proceedings of ICLR*, 2024.
- [43] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proceedings of the 20th European Conference on Computer Systems (EuroSys)*, 2025, pp. 94–109.
- [44] P. Mishra, N. Bore, B. Ramprasad, M. Thiessen, M. Gabel, A. da Silva Veith, O. Balmau, and E. de Lara, "Falcon: Live reconfiguration for stateful stream processing on the edge," in *Proceedings of the 9th ACM/IEEE Symposium on Edge Computing (SEC)*, 2024, pp. 234–248.
- [45] G. Gerganov and llama.cpp contributors, "llama.cpp: Inference of LLaMA model in pure C/C++," <https://github.com/ggerganov/llama.cpp>, 2023.
- [46] Ettus Research, "USRP B210: Usb software defined radio (sdr) datasheet," Ettus Research Product Documentation, 2014.
- [47] I. Gomez-Migueluez, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, "srsLTE: An open-source platform for LTE evolution and experimentation," in *Proceedings of the 10th ACM International Workshop on Wireless Network Testbeds, Experimental Evaluation, and Characterization (WiNTECH)*, 2016, pp. 25–32.
- [48] Q. Liang, W. A. Hanafy, N. Bashir, A. Ali-Eldin, D. Irwin, and P. Shenoy, "Dēlen: Enabling flexible and adaptive model-serving for multi-tenant edge AI," in *Proceedings of the 8th ACM/IEEE International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2023.
- [49] X. Yan and Y. Ding, "Are we there yet? a measurement study of efficiency for LLM applications on mobile devices," in *Proceedings of the 2nd International Workshop on Foundation Models for Cyber-Physical Systems and Internet of Things (FMSys)*, 2025.
- [50] S. Oh and D. Vasisht, "A call for decentralized satellite networks," in *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets)*, 2024, pp. 25–33.
- [51] W. Liu, Z. Lai, Q. Wu, H. Li, Y. Weng, W. Liu, Q. Zhang, J. Li, Y. Li, and J. Liu, "Mind the location leakage in LEO direct-to-cell satellite networks," in *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P)*, 2025.
- [52] N. Ng, A. Souza, A. Ali-Eldin, D. Irwin, D. Towsley, and P. Shenoy, "TailClipper: Reducing tail response time of distributed services through system-wide scheduling," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2024.
- [53] L. Wu, W. A. Hanafy, A. Souza, M. Satyanarayanan, D. Irwin, and P. Shenoy, "CarbonEdge: Carbon-aware edge AI inference for sustainable edge computing," *arXiv preprint arXiv:2502.14076*, 2025.
- [54] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 939–954.
- [55] B. Denby, K. Chintalapudi, R. Chandra, B. Lucia, and S. Noghbi, "Kodan: Addressing the computational bottleneck in space," in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 392–403.
- [56] T. Pfandzelter and D. Bermbach, "Komet: A serverless platform for low-earth orbit edge services," in *Proceedings of the 15th ACM Symposium on Cloud Computing (SoCC)*, 2024, pp. 866–882.
- [57] H. D. Jang, M. Varvello, A. Raman, and Y. Zaki, "From GEO to LEO: First look into starlink in-flight connectivity," in *Proceedings of the 2025 ACM Internet Measurement Conference (IMC)*, 2025, pp. 952–962.
- [58] N. Mohan, A. E. Ferguson, H. Cech, R. Bose, P. R. Renatin, M. K. Marina, and J. Ott, "A multifaceted look at Starlink performance," in *Proceedings of the ACM Web Conference (WWW)*, 2024, pp. 2723–2734.
- [59] A. Furutanpey, Q. Zhang, P. Raith, T. Pfandzelter, S. Wang, and S. Dustdar, "FOOL: Addressing the downlink bottleneck in satellite computing with neural feature compression," *IEEE Transactions on Mobile Computing*, vol. 24, no. 8, pp. 6747–6764, 2025.
- [60] Z. Cheng, B. Lucia, X. Jiang, M. Gorlatova, M. Srivastava, and G. Xing, "Geoduck: Nanosatellite constellation scheduling for low latency event detection," in *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2025, pp. 547–559.
- [61] X. Cao and X. Zhang, "SaTCP: Link-layer informed TCP adaptation for highly dynamic LEO satellite networks," in *Proceedings of IEEE INFOCOM*, 2023, pp. 1–10.
- [62] Z. Lai, H. Li, Y. Deng, Q. Wu, J. Liu, Y. Li, J. Li, L. Liu, W. Liu, and J. Wu, "StarryNet: Empowering researchers to evaluate futuristic integrated space and terrestrial networks," in *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023, pp. 1309–1324.
- [63] L. Liu, Y. Li, H. Li, J. Yang, W. Liu, J. Lan, Y. Wang, J. Li, J. Wu, Q. Wu, J. Liu, and Z. Lai, "Democratizing direct-to-cell low earth orbit satellite networks," in *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024, pp. 791–808.
- [64] J. Wu, S. Su, X. Wang, J. Zhang, and Y. Gao, "Accelerating handover in mobile satellite network," in *Proceedings of IEEE INFOCOM*, 2024, pp. 531–540.
- [65] J.-H. Lee, C. Park, S. Park, and A. F. Molisch, "Handover protocol learning for LEO satellite networks: Access delay and collision minimization," *IEEE Transactions on Wireless Communications*, vol. 23, no. 7, pp. 7624–7637, 2024.
- [66] M. Wang, S. Ding, T. Cao, Y. Liu, and F. Xu, "AsyMo: Scalable and efficient deep-learning inference on asymmetric mobile CPUs," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2021, pp. 215–228.
- [67] B. Fang, X. Zeng, and M. Zhang, "NestDNN: Resource-aware multi-tenant on-device deep learning for continuous mobile vision," in *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2018, pp. 115–127.
- [68] R. Han, Q. Zhang, C. H. Liu, G. Wang, J. Tang, and L. Y. Chen, "LegoDNN: Block-grained scaling of deep neural networks for mobile vision," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2021, pp. 406–419.
- [69] W. Zhang, Z. He, L. Liu, Z. Jia, Y. Liu, M. Gruteser, D. Raychaudhuri, and Y. Zhang, "Elf: Accelerate high-resolution mobile deep vision with content-aware parallel offloading," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2021, pp. 201–214.
- [70] E. Cuervo, A. Balasubramanian, D. ki Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "MAUI: Making smartphones last longer with code offload," in *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2010, pp. 49–62.
- [71] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "CloneCloud: Elastic execution between mobile device and cloud," in *Proceedings of the 6th European Conference on Computer Systems (EuroSys)*, 2011, pp. 301–314.
- [72] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.
- [73] Z. Li, L. Zheng, Y. Zhong, Y. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 663–679.
- [74] P. Patel, E. Choukse, C. Zhang, A. Shah, I. nigo Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proceedings of the 51st International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.
- [75] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19274–19286.
- [76] B. Ramprasad, P. Mishra, M. Thiessen, H. Chen, A. da Silva Veith, M. Gabel, O. Balmau, A. Chow, and E. de Lara, "Shepherd: Seamless stream processing on the edge," in *Proceedings of the 7th ACM/IEEE Symposium on Edge Computing (SEC)*, 2022, pp. 40–53.
- [77] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

- [78] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [79] S. Wang, R. Uргаonkar, M. Zafer, T. He, K. S. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Transactions on Networking*, vol. 27, no. 3, pp. 1272–1288, 2019.
- [80] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 10, pp. 2333–2345, 2018.